

# Python语言与系统设计

史骏

juns@hfut.edu.cn

# Python程序语法元素分析

- 程序的格式框架：缩进、注释……
- 命名与保留字：命名、变量、保留字……
- 数据类型：数据类型、字符串、整数、浮点数、列表……
- 语句与函数：赋值语句、分支语句、函数……
- 输入与输出：input()、 print()、 eval()、 print()格式化……

# Python基本数据类型

- 数字类型
- 字符串
- 列表
- 元组
- 集合
- 字典

# Python数字类型

- 整数类型
- 浮点数类型
- 复数类型
- 布尔类型
- 数值运算操作符
- 数值运算函数

# 浮点数类型

浮点数间运算存在不确定尾数，不是bug

```
>>> 0.1+0.3
0.4
>>> 0.1+0.2
0.30000000000000004
>>>
```

不确定尾数

# 浮点数类型

浮点数间运算存在不确定尾数

```
>>> 0.1+0.2==0.3
False
>>> round(0.1+0.2,1) == 0.3
True
>>>
```

# 浮点数类型

浮点数间运算存在不确定尾数

- round(x, d)：对x四舍五入， d是小数截取位数
- 浮点数间运算与比较用round()函数辅助
- 不确定尾数一般发生在$10^{-16}$左右， round()十分有效

# 浮点数类型

浮点数可以采用科学计数法表示

- 使用字母e或E作为幂的符号，以10为基数，格式如下：

  **<a>e<b>** 表示 **$a*10^b$**

- 例如： **4.3e-3** 值为0.0043 **9.6E5** 值为960000.0

# 布尔类型

## 与数学中实数的概念一致

- Python中的布尔类型只有两种值：True 和 False
- False可以是数值为0、对象为None或者是序列中的空字符串、空列表、空元组

type() 函数可以用来查询变量所指的对象类型

```
>>> a, b, c, d = 20, 5.5, True, 4+3j
>>> print(type(a), type(b), type(c), type(d))
<class 'int'> <class 'float'> <class 'bool'> <class 'complex'>
>>>
```

# 数值运算操作符

操作符是完成运算的一种符号体系

| 操作符及使用 | 描述 |
| --- | --- |
| x + y | 加， x与y之和 |
| x – y | 减， x与y之差 |
| x * y | 乘， x与y之积 |
| x / y | 除， x与y之商 10/3结果是3.3333333333333335 |
| x // y | 整数除， x与y之整数商 10//3结果是3 |

# 数值运算操作符

操作符是完成运算的一种符号体系

| 操作符及使用 | 描述 |
| --- | --- |
| + x | x本身 |
| - x | x的负值 |
| x % y | 余数，模运算 10%3结果是1 |
| x ** y | 幂运算， x的y次幂， $x^y$ |
| | 当y是小数时，开方运算10**0.5结果是 $\sqrt{10}$ |

# 数值运算操作符

## 二元操作符有对应的增强赋值操作符

| 操作符及使用 | 描述 |
| --- | --- |
| x **op**= y | 即 x = x **op** y，其中， **op**为二元操作符 |
| | x += y x -= y x *= y x /= y<br>x //= y x %= y x **= y |
| | **>>> x = 3.1415**<br>**>>> x **= 3 # 与 x = x **3 等价**<br>**31.003533398375005** |

# 数字类型的关系

类型间可进行混合运算，生成结果为"最宽"类型

- 三种类型存在一种逐渐"扩展"或"变宽"的关系：

  **整数 -> 浮点数 -> 复数**

- 例如： 123 + 4.0 = 127.0 (整数+浮点数 = 浮点数)

# Python数字类型

- 整数类型
- 浮点数类型
- 复数类型
- 布尔类型
- 数值运算操作符
- 数值运算函数

# 数值运算函数

## 一些以函数形式提供的数值运算功能

| 函数及使用 | 描述 |
| --- | --- |
| abs(x) | 绝对值， x的绝对值<br>abs(-10.01) 结果为 10.01 |
| divmod(x,y) | 商余， (x//y, x%y)，同时输出商和余数<br>divmod(10, 3) 结果为 (3, 1) |
| pow(x, y[, z]) | 幂余， (x**y)%z， [..]表示参数z可省略<br>pow(3, pow(3, 99), 10000) 结果为 4587 |

# 数值运算函数

## 一些以函数形式提供的数值运算功能

| 函数及使用 | 描述 |
| --- | --- |
| round(x[, d]) | 四舍五入， d是保留小数位数，默认值为0<br>round(-10.123, 2) 结果为 -10.12 |
| max($x_1$,$x_2$, ... ,$x_n$) | 最大值，返回$x_1$,$x_2$, ... ,$x_n$中的最大值， n不限<br>max(1, 9, 5, 4, 3) 结果为 9 |
| min($x_1$,$x_2$, ... ,$x_n$) | 最小值，返回$x_1$,$x_2$, ... ,$x_n$中的最小值， n不限<br>min(1, 9, 5, 4, 3) 结果为 1 |

# 数值运算函数

## 一些以函数形式提供的数值运算功能

| 函数及使用 | 描述 |
| --- | --- |
| int(x) | 将x变成整数，舍弃小数部分<br>int(123.45) 结果为123； int("123") 结果为123 |
| float(x) | 将x变成浮点数，增加小数部分<br>float(12) 结果为12.0； float("1.23") 结果为1.23 |
| complex(x) | 将x变成复数，增加虚数部分<br>complex(4) 结果为 4 + 0j |

# Python数字类型

- 整数类型
- 浮点数类型
- 复数类型
- 布尔类型
- 数值运算操作符
- 数值运算函数

# 字符串

由0个或多个字符组成的有序字符序列

- 字符串由一对单引号或一对双引号表示

  **"请输入带有符号的温度值: "**或者 **'C'**

- 字符串是字符的有序序列，可以对其中的字符进行索引

  **"请"** 是 **"请输入带有符号的温度值: "** 的第0个字符

# 字符串的序号

## 正向递增序号 和 反向递减序号



# 字符串的使用

使用 [] 获取字符串的一个或多个字符

- 索引：返回字符串中单个字符　　**<字符串>[M]**

  **TempStr[-1]**

- 切片：返回字符串中一段字符子串　　**<字符串>[M: N]**

  **TempStr[0:-1]**

# 字符串切片高级用法

使用[M: N: K]根据步长对字符串切片

- <字符串>[M: N]， M缺失表示**至开头**， N缺失表示**至结尾**

  **"〇一二三四五六七八九十"[:3]** 结果是 **"〇一二"**

- <字符串>[M: N: K]，根据步长K对字符串切片

  **"〇一二三四五六七八九十"[1:8:2]** 结果是 **"一三五七"**

  **"〇一二三四五六七八九十"[::-1]** 结果是 **"十九八七六五四三二一〇"**

# 字符串操作符

由0个或多个字符组成的有序字符序列

| 操作符及使用 | 描述 |
| --- | --- |
| x + y | 连接两个字符串x和y |
| n * x 或 x * n | 复制n次字符串x |
| x in s | 如果x是s的子串，返回True，否则返回False |

# 字符串处理函数

## 一些以函数形式提供的字符串处理功能

| 函数及使用 | 描述 |
| --- | --- |
| len(x) | 长度，返回字符串x的长度<br>len("一二三456") 结果为 6 |
| str(x) | 任意类型x所对应的字符串形式<br>str(1.23)结果为'1.23' str([1,2])结果为'[1,2]' |
| hex(x) **或** oct(x) | 整数x的十六进制或八进制小写形式字符串<br>hex(425)结果为'0x1a9' oct(425)结果为'0o651' |

# 字符串处理方法

## 一些以方法形式提供的字符串处理功能

| 方法及使用 | 描述 |
| --- | --- |
| str.lower() **或**<br>str.upper() | 返回字符串的副本，全部字符小写/大写<br>"AbCdEfGh"**.lower()** 结果为 'abcdefgh' |
| str.split(sep=None) | 返回一个列表，由str根据sep被分隔的部分组成<br>"A,B,C"**.split(",")** 结果为 ['A','B','C'] |
| str.count(sub) | 返回子串sub在str中出现的次数<br>"an apple a day"**.count("a")** 结果为 4 |

# 字符串处理方法

## 一些以方法形式提供的字符串处理功能

| 方法及使用 | 描述 |
| --- | --- |
| str.replace(old, new) | 返回字符串str副本，所有old子串被替换为new<br>"python".replace("n","n123.io") 结果为<br>'python123.io' |
| str.center(width[,fillchar]) | 字符串str根据宽度width居中， fillchar可选<br>"python".center(20,"=") 结果为<br>'=======python=======' |

# 字符串处理方法

## 一些以方法形式提供的字符串处理功能

| 方法及使用 | 描述 |
| --- | --- |
| str.strip(chars) | 从str中去掉在其左侧和右侧chars中列出的字符<br>"= python= ".strip(" =np") 结果为<br>'ytho' |
| str.join(iter) | 在iter变量除最后元素外每个元素后增加一个str<br>",".join("12345") 结果为<br>"1,2,3,4,5" #主要用于字符串分隔等 |

# 字符串类型的格式化

格式化是对字符串进行格式表达的方式

- 字符串格式化使用.format()方法，用法如下：

**<模板字符串>.format(<逗号分隔的参数>)**

# 字符串类型的格式化

槽

"{ }:计算机{ }的CPU占用率为{ }% ".format("2018-10-10","C",10)

0 1 2

0 1 2

字符串中槽{}的默认顺序

format()中参数的顺序

# 字符串类型的格式化

槽

"{1}:计算机{0}的CPU占用率为{2}% ".format("2018-10-10","C",10)

```
>>> "{1}:计算机{0}的CPU占用率为{2}%".format("2018-10-10","C",10)
'C:计算机2018-10-10的CPU占用率为10%'
>>>
```

# 算术运算符

| 运算符及使用 | 描述 |
| --- | --- |
| x + y | 加， x与y之和 |
| x – y | 减， x与y之差 |
| x * y | 乘， x与y之积 |
| x / y | 除， x与y之商，10/3结果是3.3333333333333335 |
| x % y | 取模：除法运算求余数, 34%10结果是4 |
| x ** y | 幂，2**4结果是16 |
| x // y | 整数除， x与y之整数商 10//3结果是3 |

# 比较运算符

| 运算符 | 描述 |
| --- | --- |
| == | 等于，结果True或者False |
| != | 不等于，结果True或者False |
| > | 大于，结果True或者False |
| < | 小于，结果True或者False |
| >= | 大于等于，结果True或者False |
| <= | 小于等于，结果True或者False |

# 逻辑运算符

| 运算符 | 表达式 | 描述 |
| --- | --- | --- |
| **and** | x and y | 两边都是True时返回True, 否则返回False |
| **or** | x or y | 两边都是False时返回False, 否则返回True |
| **not** | not x | 表达式取反，返回值与原值相反 |

# 二元运算符

| 二元运算符及使用 | 描述 |
| --- | --- |
| x **op**= y | 即 x = x **op** y，其中， **op**为二元操作符 |
| | x += y x -= y x *= y x /= y<br>x //= y x %= y x **= y |
| | **>>> x = 3.1415**<br>**>>> x **= 3 # 与 x = x **3 等价**<br>**31.003533398375005** |

# 成员运算符

| 运算符 | 表达式 | 描述 |
| --- | --- | --- |
| **in** | 如果在指定的序列中找到值返回 True，否则返回 False。 | x 在 y 序列中，如果 x 在 y 序列中返回 True |
| **not in** | 如果在指定的序列中没有找到值返回 True，否则返回 False。 | x 不在 y 序列中，如果 x 不在 y 序列中返回 True |

# 身份运算符

| 运算符 | 表达式 | 描述 |
| --- | --- | --- |
| **is** | is 是判断两个标识符是不是引用自一个对象 | **x is y**, 类似 **id(x) == id(y)** , 如果引用的是同一个对象则返回 True，否则返回 False |
| **is not** | is not 是判断两个标识符是不是引用自不同对象 | **x is not y** ， 类似 **id(a) != id(b)**。如果引用的不是同一个对象则返回结果 True，否则返回 False |

# 运算符优先级

| 运算符 | 描述 |
| --- | --- |
| ** | 指数 (最高优先级) |
| ~ + - | 按位翻转, 一元加号和减号 (最后两个的方法名为 +@ 和 -@) |
| * / % // | 乘，除，求余数和取整除 |
| + - | 加法减法 |
| >> << | 右移，左移运算符 |
| & | 位 'AND' |
| ^ \| | 位运算符 |
| <= < > >= | 比较运算符 |
| == != | 等于运算符 |
| = %= /= //= -= += *= **= | 赋值运算符 |
| is is not | 身份运算符 |
| in not in | 成员运算符 |
| not and or | 逻辑运算符 |






史骏
juns@hfut.edu.cn